

DTS103TC

Data Structure and Algorithms

Lab 10: Maximum Flow

Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

Learning outcome

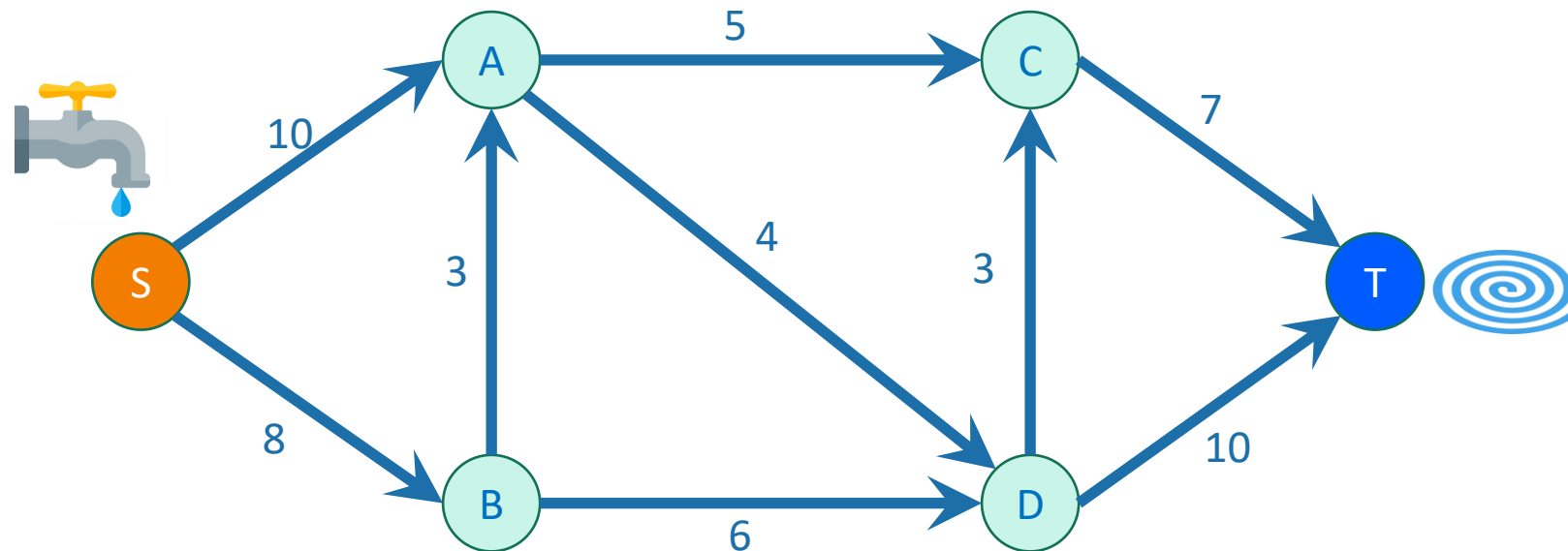
- Flow Network Representation
- Finding Augmenting Paths (BFS)
- Ford-Fulkerson Algorithm
- Bipartite Matching

Flow Network

- Flow Network:

- Graphs are directed
- Edges have "capacities" (weights)
- Two distinguished vertices, namely
 - "source" vertex S has only outgoing edges
 - "sink" vertex T has only incoming edges

Graphs can be used to model the flow of resources in real-world systems, such as transportation networks, communication systems, and allocation problems.

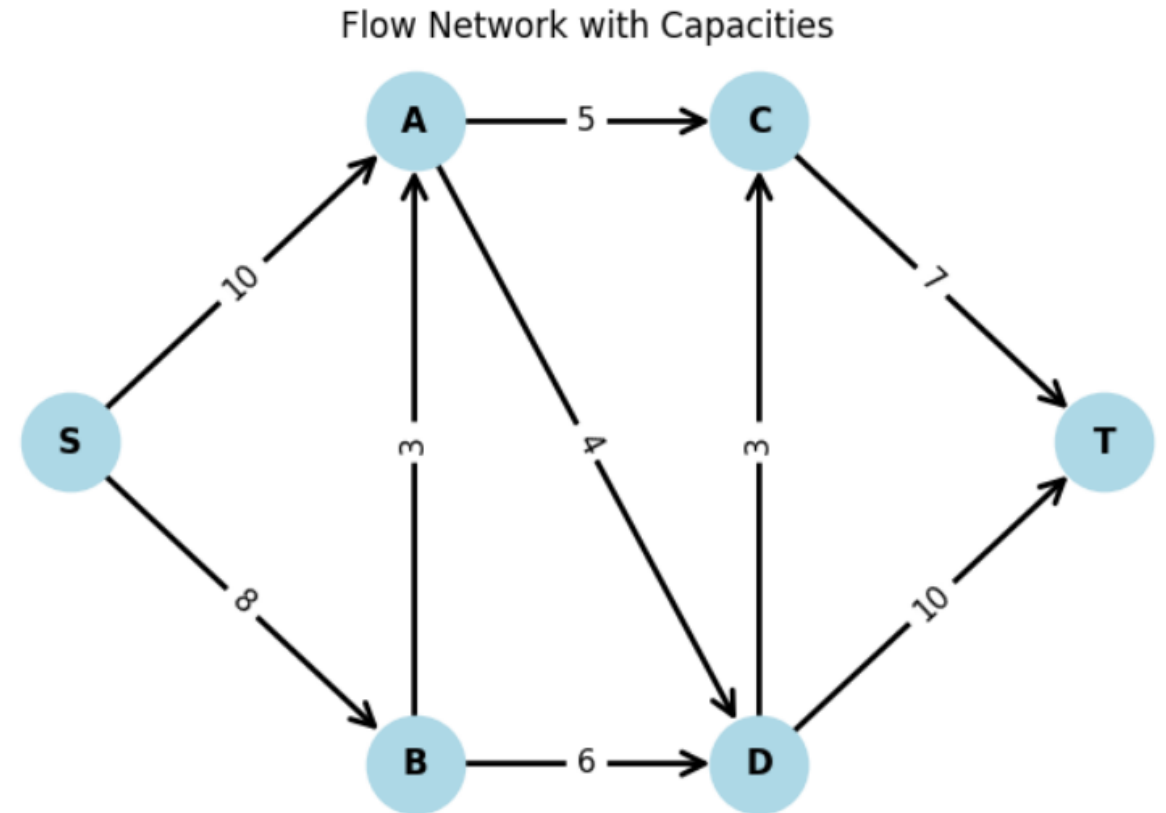


Flow Network Representation

```
flow_network = {  
    'S': {'A': 10, 'B': 8},  
    'A': {'C': 5, 'D': 4},  
    'B': {'A': 3, 'D': 6},  
    'C': {'T': 7},  
    'D': {'C': 3, 'T': 10},  
    'T': {}  
}
```

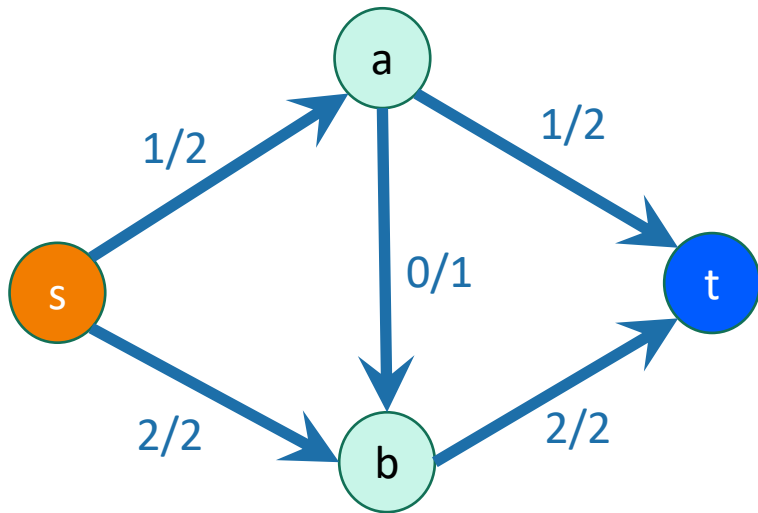
■ Interpretation

- S → A : 10 means the capacity from S to A is 10
- D → T : 10 means at most 10 units of flow can go from D to T

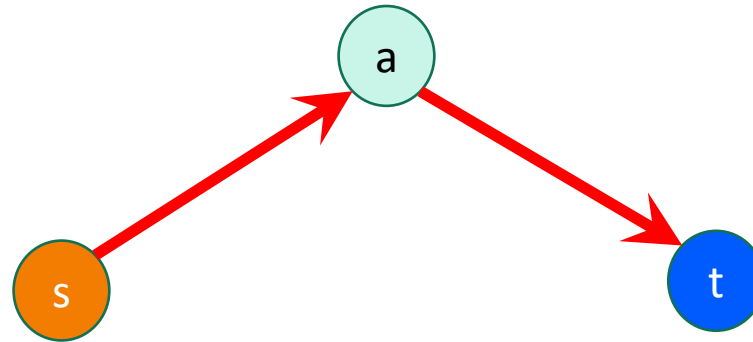


Finding an Augmenting Path

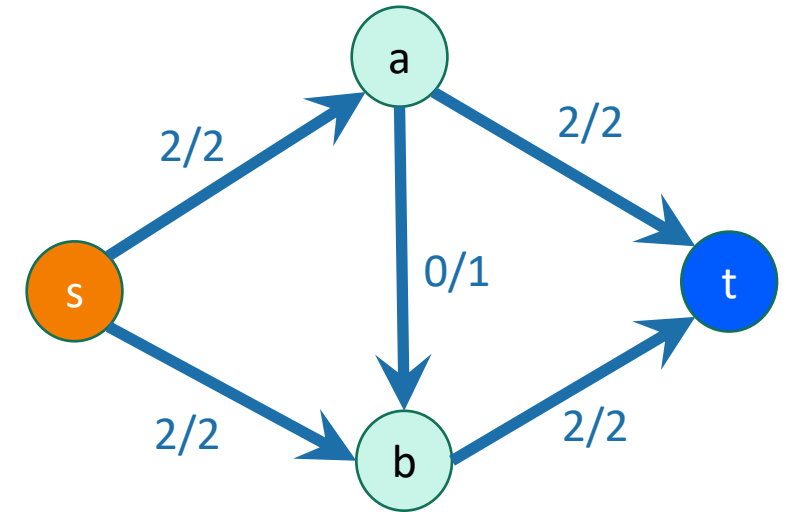
- An **augmenting path** is a path from the source S to the sink T where every edge has **positive residual capacity**.



A network with a flow of value 3



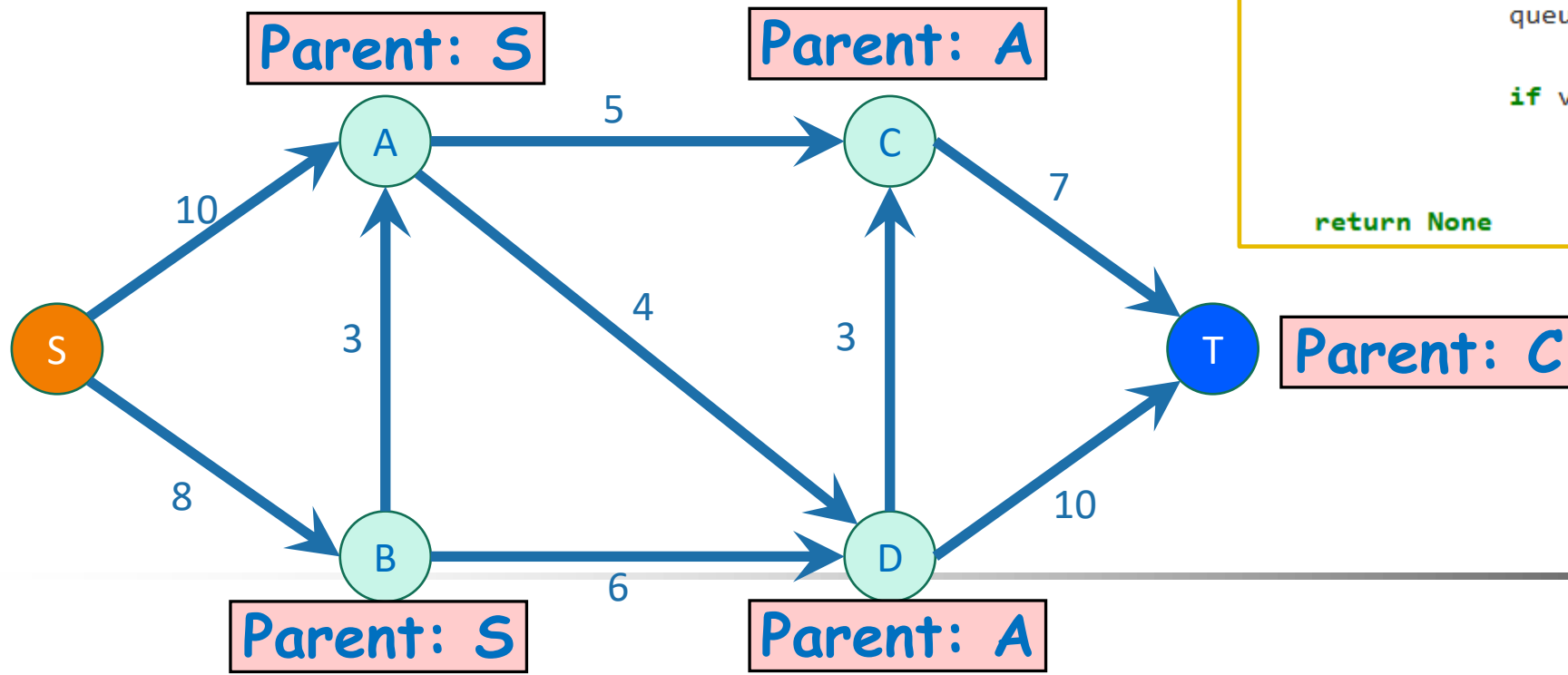
Augmenting path



The flow value is increased to 4

Using BFS to Find a Path

- BFS helps us:
 - explore nodes level by level
 - find a path from S to T
 - record the path using a parent dictionary

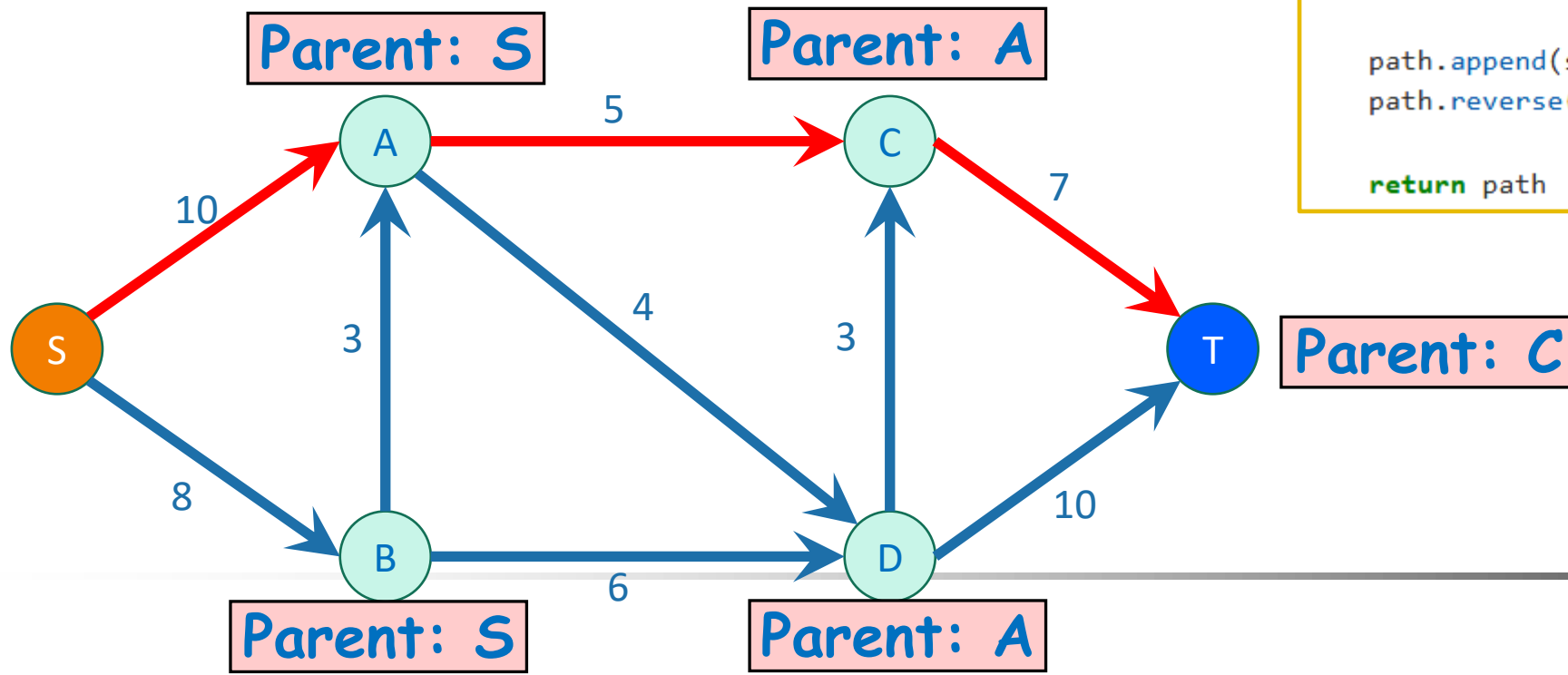


```
def bfs_augmenting_path(residual, source, sink):  
    visited = set([source])  
    parent = {}  
    queue = deque([source])  
  
    while queue:  
        u = queue.popleft()  
  
        for v in residual[u]:  
            if v not in visited and residual[u][v] > 0:  
                visited.add(v)  
                parent[v] = u  
                queue.append(v)  
  
            if v == sink:  
                return parent  
  
    return None
```

Using BFS to Find a Path

- Reconstruct the augmenting path from S to T using the parent dictionary.

```
def reconstruct_path(parent, source, sink):  
    if parent is None:  
        return None  
  
    path = []  
    current = sink  
  
    while current != source:  
        path.append(current)  
        current = parent[current]  
  
    path.append(source)  
    path.reverse()  
  
    return path
```



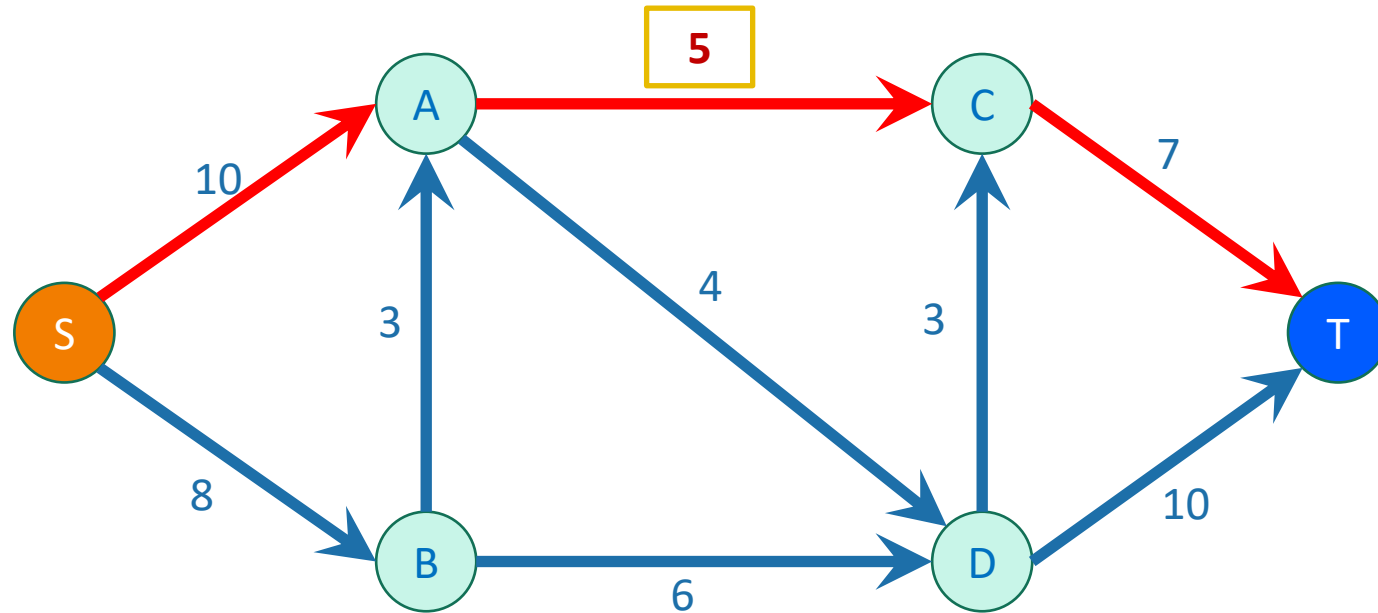
Ford-Fulkerson Algorithm

- The Ford-Fulkerson algorithm computes the maximum flow from source S to sink T .
- The main idea is:
 - Use BFS to find an augmenting path ✓
 - Find the minimum residual capacity along the path (bottleneck capacity)
 - Update the residual graph
 - Repeat until no augmenting path exists

Ford-Fulkerson Algorithm

- Find the minimum residual capacity along the path (bottleneck capacity)

```
# find bottleneck
path_flow = float('inf')
for i in range(len(path) - 1):
    u = path[i]
    v = path[i + 1]
    path_flow = min(path_flow, residual[u][v])
```



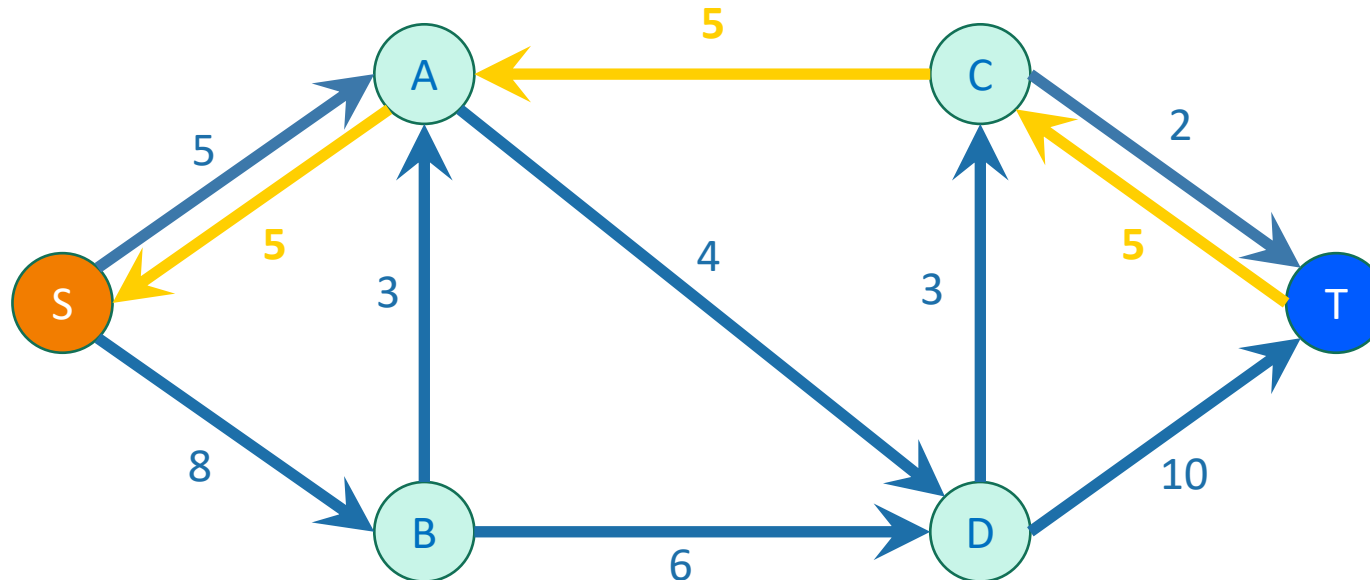
Ford-Fulkerson Algorithm

- Update the residual graph

Residual graph initialization
Before the while loop

```
# initialize residual graph
residual = {u: dict(flow_network[u]) for u in flow_network}

# make sure all nodes exist in residual graph
for u in flow_network:
    for v in flow_network[u]:
        if v not in residual:
            residual[v] = {}
        if u not in residual[v]:
            residual[v][u] = 0
```



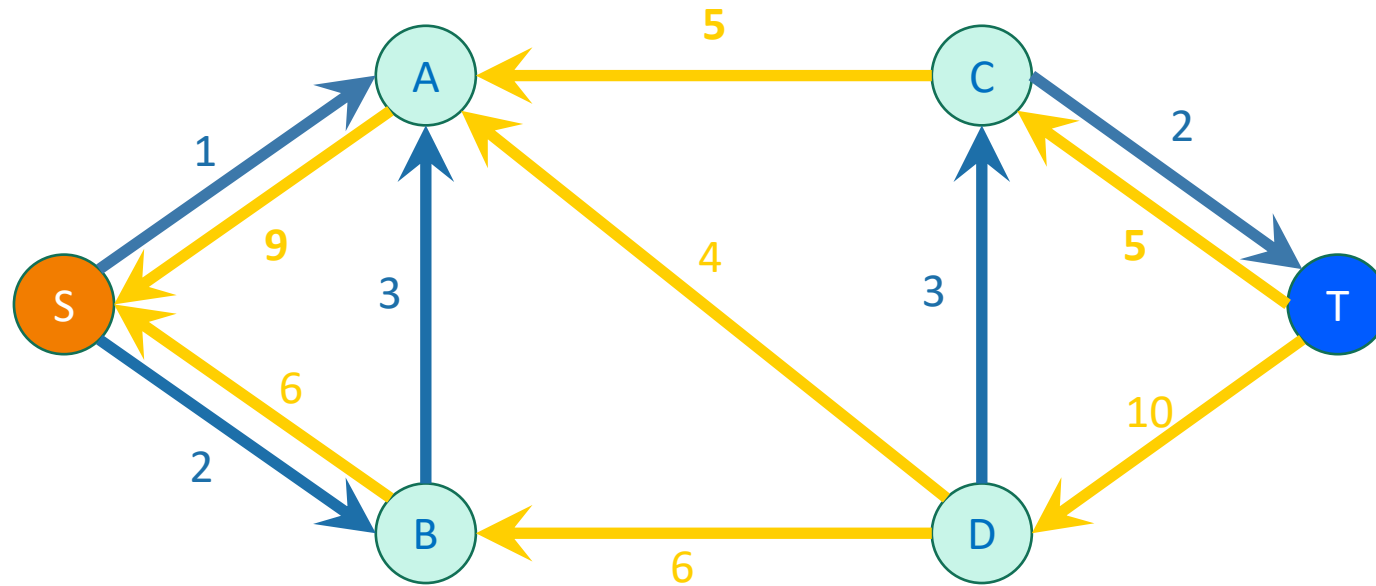
Residual graph update
Inside the while loop

```
# update residual graph
for i in range(len(path) - 1):
    u = path[i]
    v = path[i + 1]

    residual[u][v] -= path_flow
    residual[v][u] += path_flow
```

Ford-Fulkerson Algorithm

- Repeat until no augmenting path exists

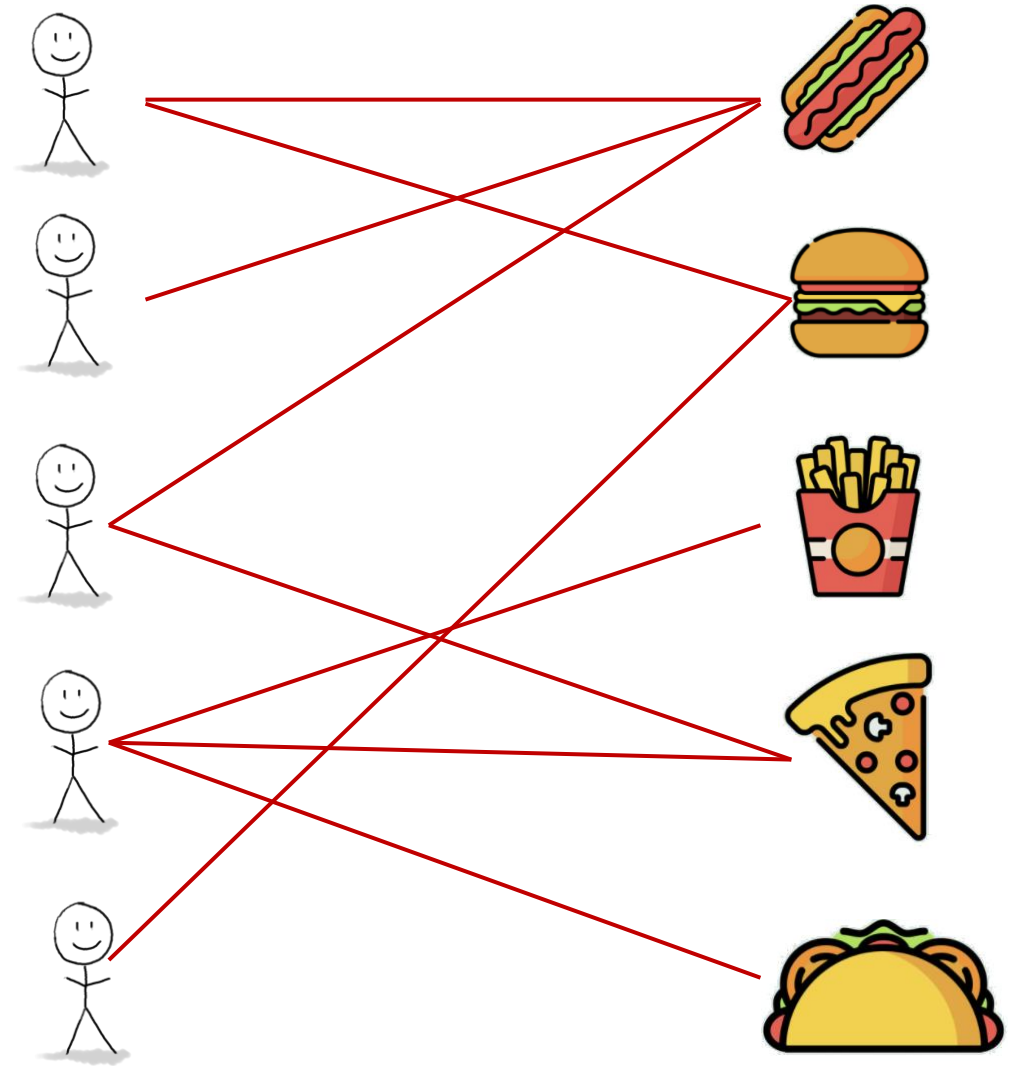


Augmenting path: ['S', 'A', 'C', 'T'] flow added: 5
Augmenting path: ['S', 'A', 'D', 'T'] flow added: 4
Augmenting path: ['S', 'B', 'D', 'T'] flow added: 6
Maximum flow: 15

Maximum Bipartite Matching

- Different students want different types of food
- How can we make as many students as possible happy?

```
students = ['A', 'B', 'C', 'D', 'E']  
food = ['1', '2', '3', '4', '5']  
  
edges = [  
    ('A', '1'), ('A', '2'),  
    ('B', '1'),  
    ('C', '1'), ('C', '4'),  
    ('D', '3'), ('D', '4'), ('D', '5'),  
    ('E', '2')  
]
```



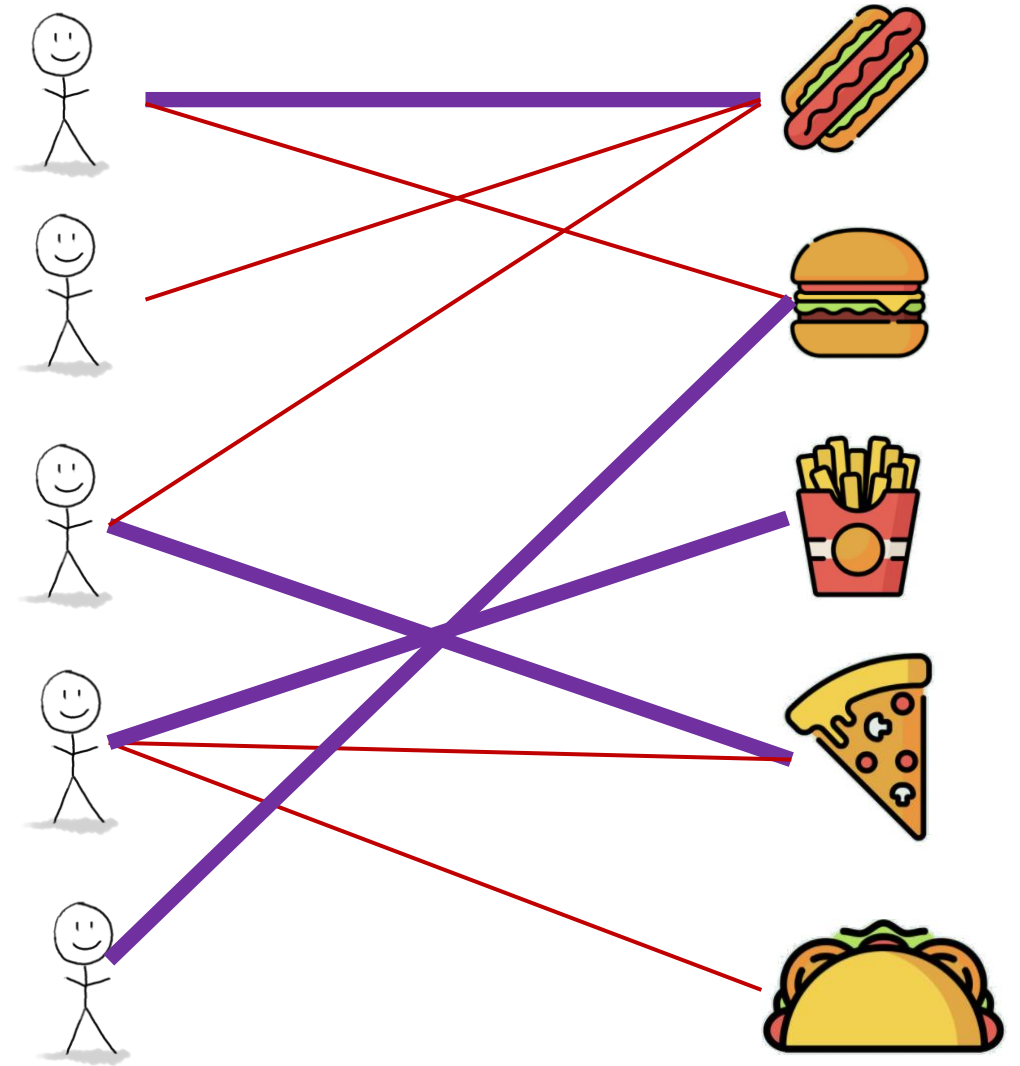
Students

Food

Maximum Bipartite Matching

- Different students want different types of food
- How can we make as many students as possible happy?

```
students = ['A', 'B', 'C', 'D', 'E']  
food = ['1', '2', '3', '4', '5']  
  
edges = [  
    ('A', '1'), ('A', '2'),  
    ('B', '1'),  
    ('C', '1'), ('C', '4'),  
    ('D', '3'), ('D', '4'), ('D', '5'),  
    ('E', '2')  
]
```

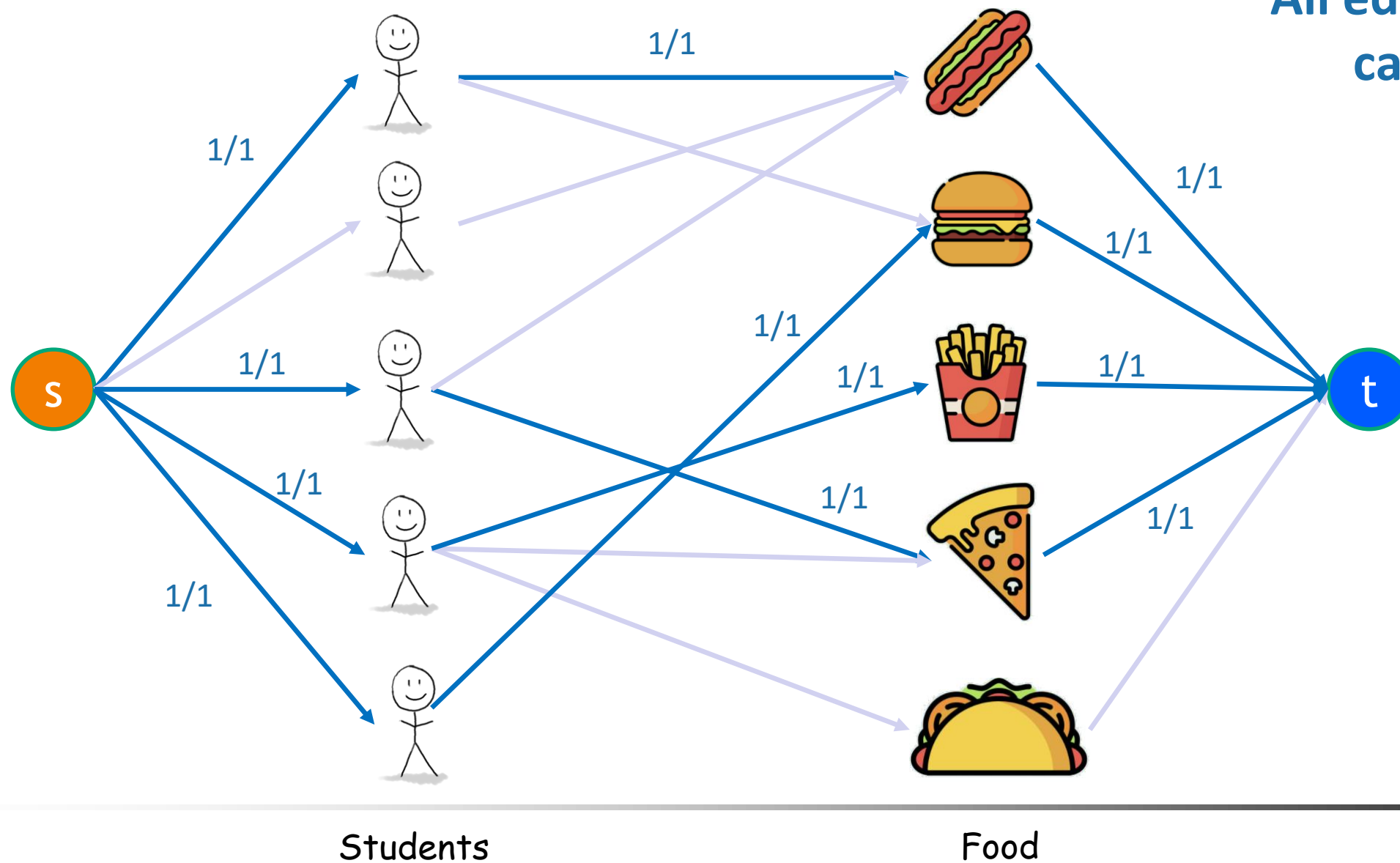


Students

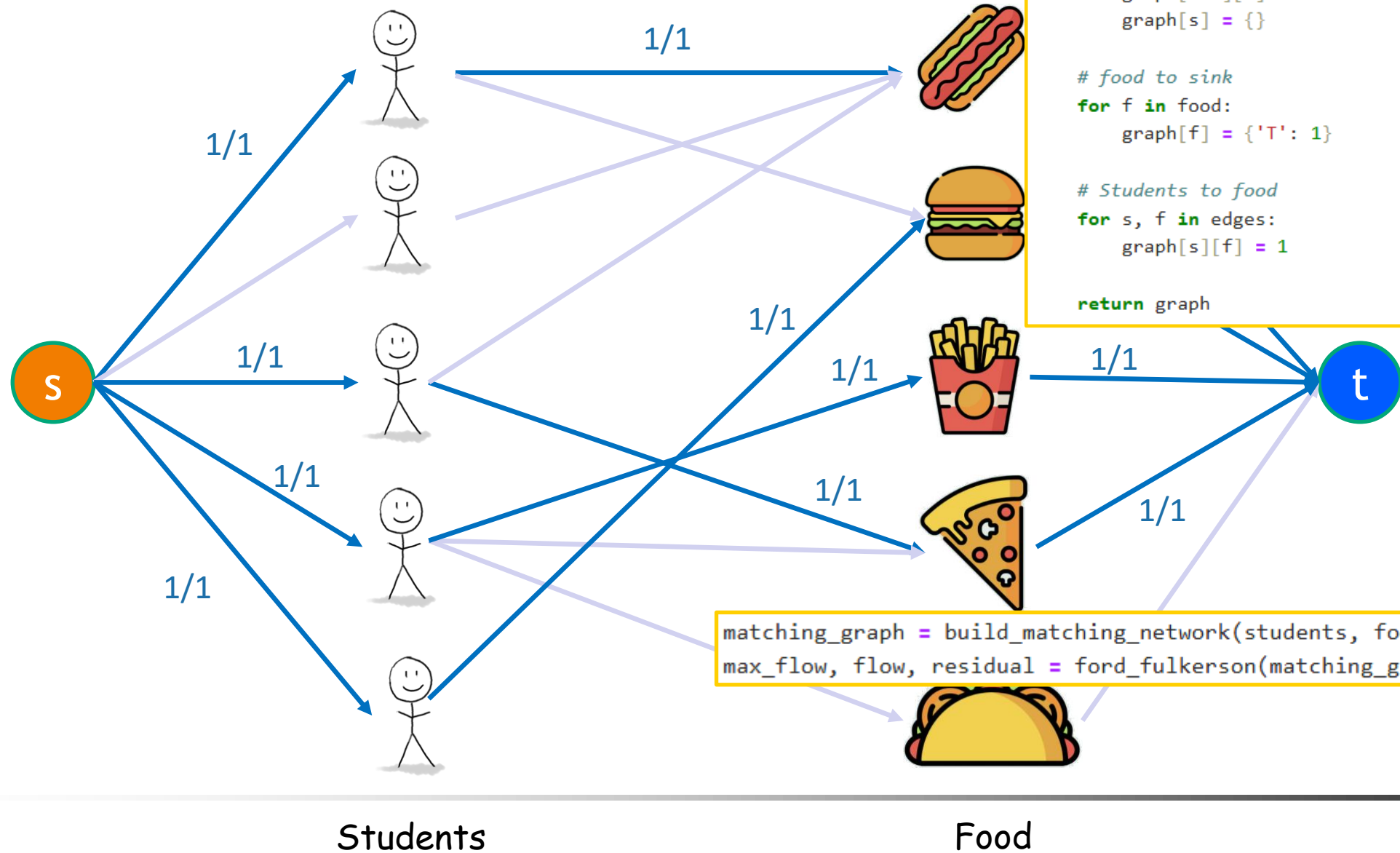
Food

Solution via max flow

All edges have capacity 1.



Solution via max flow



```
def build_matching_network(students, food, edges):  
    graph = {'S': {}, 'T': {}}  
  
    # Source to students  
    for s in students:  
        graph['S'][s] = 1  
        graph[s] = {}  
  
    # food to sink  
    for f in food:  
        graph[f] = {'T': 1}  
  
    # Students to food  
    for s, f in edges:  
        graph[s][f] = 1  
  
    return graph
```

```
matching_graph = build_matching_network(students, food, edges)  
max_flow, flow, residual = ford_fulkerson(matching_graph, 'S', 'T')
```

Learning outcome

- Flow Network Representation
- Finding Augmenting Paths (BFS)
- Ford-Fulkerson Algorithm
- Bipartite Matching